

ShieldFi — Decentralized Insurance Pool

Final Project Report

Students

Tlegen Tolegenuly, Sanzhar Sagatov, Kasymov Beksultan

Project Option

Option E — Decentralized Insurance Pool

Network

Ethereum Sepolia

1. Introduction

ShieldFi is a decentralized insurance protocol built on Ethereum Sepolia that allows users to:

- provide liquidity into an insurance vault,
- purchase insurance policies,
- process claims using oracle-based conditions,
- participate in DAO governance,
- analyze protocol activity through indexed blockchain data.

The goal of the project is to demonstrate a complete full-stack Web3 architecture that combines:

- Solidity smart contracts,
- ERC4626 tokenized vaults,
- DAO governance,
- The Graph subgraphs,
- React frontend integration,
- automated testing and security analysis.

2. System Architecture

Core Components

The protocol consists of the following modules:

Component	Description
InsuranceVault	ERC4626 vault for underwriting liquidity
PolicyManager	Handles insurance policy purchases
ClaimProcessor	Processes claims using Chainlink price data
ShieldGovToken	Governance token with voting power
ShieldProtocolGovernor	DAO governance contract
ProtocolTreasury	Treasury management contract
Subgraph	Indexes protocol events for analytics
React Frontend	User interface for protocol interaction

Smart Contract Structure

Contracts are organized into modular directories:

- contracts/vault/
- contracts/policy/
- contracts/claims/
- contracts/governance/
- contracts/token/
- contracts/treasury/
- contracts/interfaces/

This modular architecture improves readability, maintainability, and scalability.

3. Smart Contracts

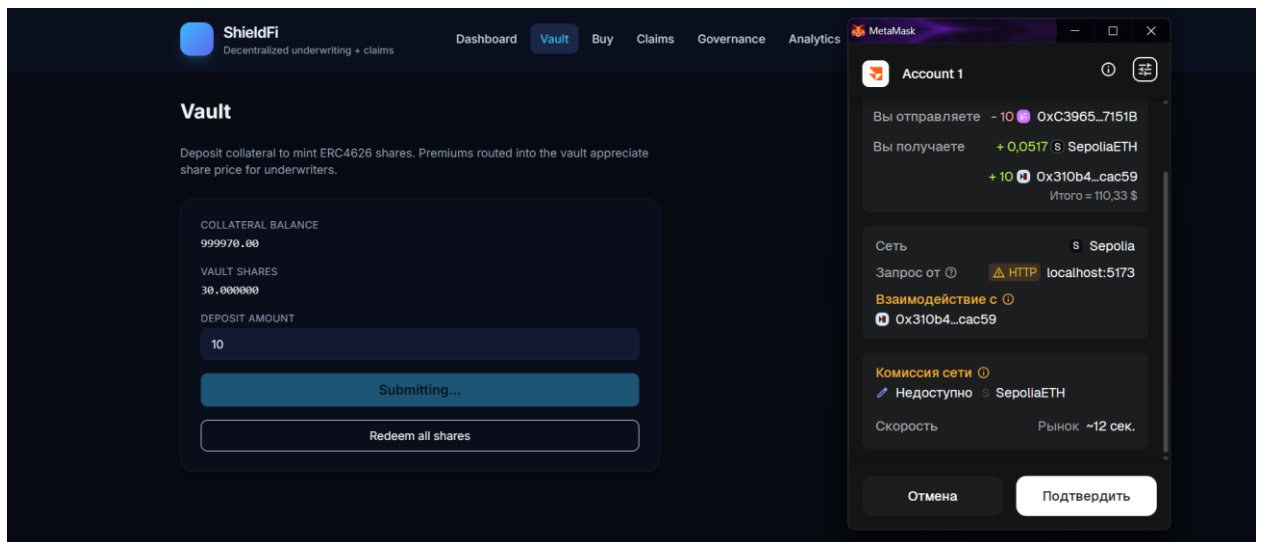
InsuranceVault (ERC4626)

The InsuranceVault contract is implemented using the ERC4626 tokenized vault standard.

Features:

- deposit collateral,
- mint vault shares,
- redeem shares,
- yield accumulation from premiums,
- underwriting liquidity management.

The vault allows liquidity providers to participate in protocol revenue generation.



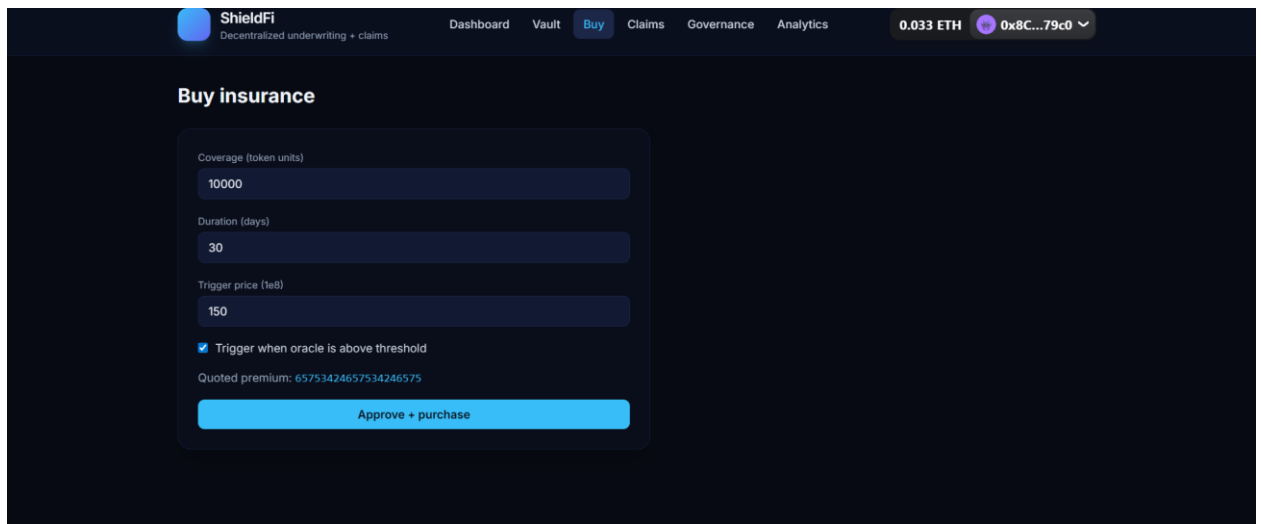
PolicyManager

The PolicyManager contract allows users to purchase insurance coverage.

Features:

- policy creation,
- premium calculation,
- expiration management,
- trigger condition configuration.

Premiums are routed into the vault.



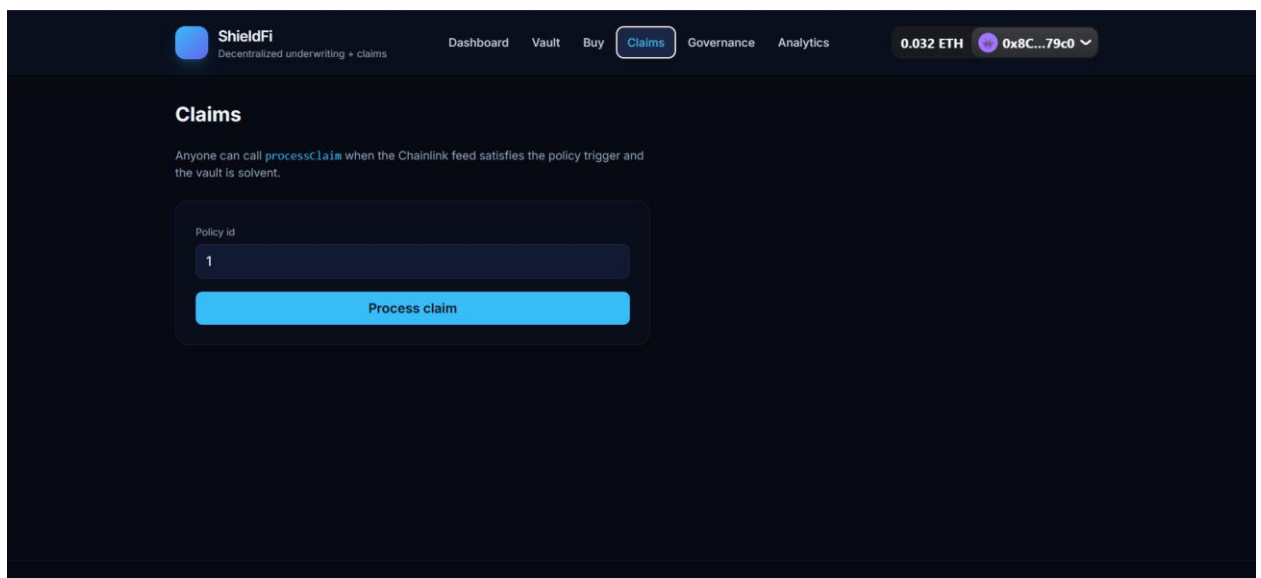
The screenshot shows the 'Buy insurance' page of the ShieldFi application. The top navigation bar includes the ShieldFi logo, the tagline 'Decentralized underwriting + claims', and links to Dashboard, Vault, Buy, Claims, Governance, and Analytics. On the right, it displays a balance of 0.033 ETH and a wallet address 0x8C...79c0. The main content area is titled 'Buy insurance' and contains a form with the following fields: 'Coverage (token units)' with a value of 10000, 'Duration (days)' with a value of 30, and 'Trigger price (1e8)' with a value of 150. There is a checked checkbox for 'Trigger when oracle is above threshold' and a 'Quoted premium: 65753424657534246575'. A blue button at the bottom of the form is labeled 'Approve + purchase'.

ClaimProcessor

The ClaimProcessor contract validates claims using Chainlink oracle price data.

Features:

- oracle-based validation,
- automated payout logic,
- vault solvency checks,
- secure payout execution.



The screenshot shows the 'Claims' page of the ShieldFi application. The top navigation bar is similar to the previous one, but the 'Claims' link is highlighted. The main content area is titled 'Claims' and includes a sub-header: 'Anyone can call `processClaim` when the Chainlink feed satisfies the policy trigger and the vault is solvent.' Below this, there is a form with a 'Policy id' field containing the value '1' and a blue button labeled 'Process claim'.

Governance System

Governance is implemented using:

- ERC20Votes,
- OpenZeppelin Governor,

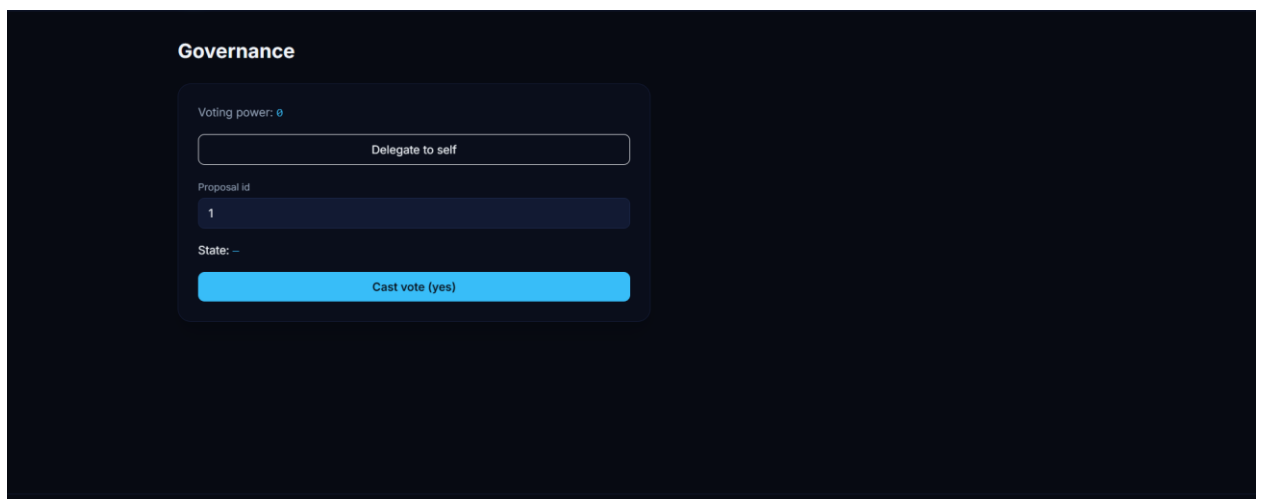
- proposal lifecycle,
- on-chain voting.

Users can:

- delegate voting power,
- vote on proposals,
- participate in protocol upgrades.

Governance Lifecycle

1. Proposal creation
2. Voting delay
3. Active voting period
4. Vote counting
5. Proposal execution



The screenshot displays a dark-themed web interface for a governance system. At the top, the word "Governance" is centered. Below it, a form is visible with the following elements: "Voting power: 0" in light blue text; a button labeled "Delegate to self"; a label "Proposal id" above a text input field containing the number "1"; a label "State: -" above a large, bright blue button labeled "Cast vote (yes)".

4. Frontend Application

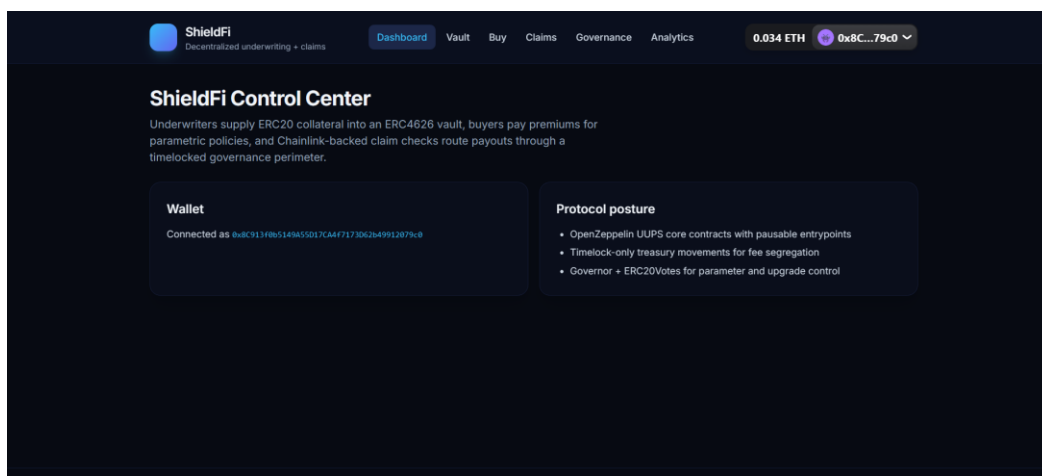
The frontend is built using:

- React,
- TypeScript,
- Vite,
- wagmi,
- viem.

Main Features

- MetaMask integration,
- vault deposits,
- policy purchasing,
- claim processing,
- DAO voting,
- analytics dashboard.

The frontend includes improved error handling for blockchain transaction failures.



5. Subgraph and Analytics

The Graph Studio was used to deploy a custom subgraph.

Indexed contracts:

- InsuranceVault,
- PolicyManager,
- ClaimProcessor,
- ProtocolTreasury.

The subgraph indexes:

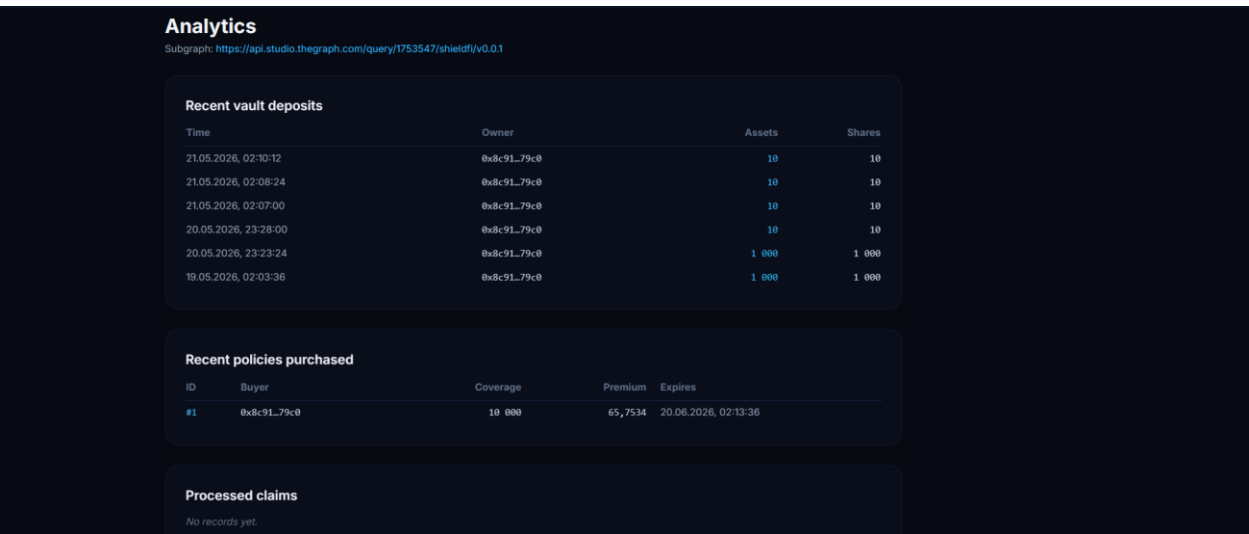
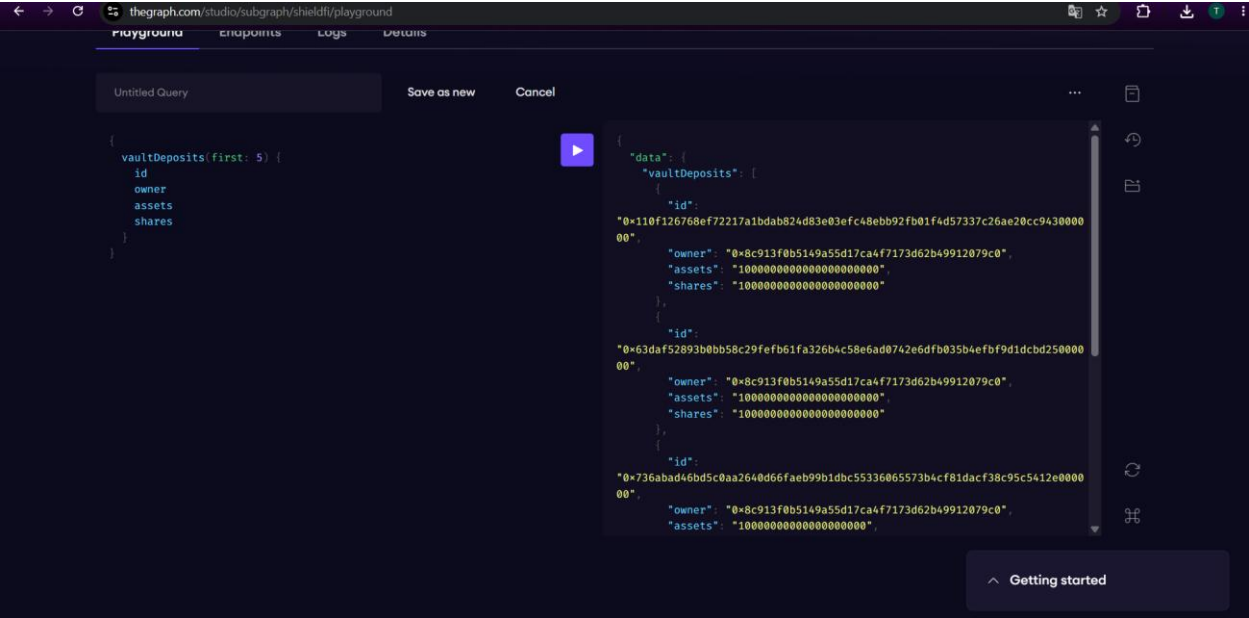
- deposits,
- withdrawals,
- policy purchases,
- processed claims.

GraphQL Endpoint

The deployed subgraph endpoint:

<https://api.studio.thegraph.com/query/1753547/shieldfi/v0.0.1>

The analytics page retrieves indexed blockchain data through GraphQL queries.



6. Testing

A comprehensive testing suite was implemented using Foundry.

Test Categories

Test Type	Purpose
Unit Tests	Validate individual functions
Integration Tests	Validate protocol interactions

Test Type	Purpose
Fuzz Tests	Randomized input validation
Invariant Tests	Protocol invariant validation
Fork Tests	Real network simulation
Governance Tests	Proposal lifecycle validation

Results

- 241 tests passed
- 0 failed

```
[PASS] invariant_vaultSolvent() (runs: 256, calls: 128000, reverts: 0)

+-----+-----+-----+-----+-----+
| Contract | Selector | Calls | Reverts | Discards |
+-----+-----+-----+-----+-----+
| VaultHandler | deposit | 63714 | 0 | 0 |
+-----+-----+-----+-----+-----+
| VaultHandler | withdraw | 64286 | 0 | 0 |
+-----+-----+-----+-----+-----+

Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 20.73s (90.79s CPU time)

Ran 12 tests for test/CoverageBoost.t.sol:TreasuryCoverageTest
[PASS] test_treasuryConstructorZeroTimelock() (gas: 58147)
[PASS] test_treasuryReceivesNative() (gas: 19081)
[PASS] test_treasuryTimelockAddress() (gas: 7621)
[PASS] test_treasuryWithdrawERC20HappyPath() (gas: 317554)
[PASS] test_treasuryWithdrawERC20RevertsNonTimelock() (gas: 18878)
[PASS] test_treasuryWithdrawERC20RevertsZeroAmount() (gas: 20586)
[PASS] test_treasuryWithdrawERC20RevertsZeroTo() (gas: 18500)
[PASS] test_treasuryWithdrawERC20RevertsZeroToken() (gas: 18445)
[PASS] test_treasuryWithdrawNativeHappyPath() (gas: 50189)
[PASS] test_treasuryWithdrawNativeRevertsNonTimelock() (gas: 16624)
[PASS] test_treasuryWithdrawNativeRevertsZeroAmount() (gas: 16425)
[PASS] test_treasuryWithdrawNativeRevertsZeroTo() (gas: 14273)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 20.74s (727.20µs CPU time)

Ran 18 test suites in 20.74s (44.59s CPU time): 241 tests passed, 0 failed, 0 skipped (241 total tests)
```

7. Security Analysis

Slither Static Analysis

Slither was used for smart contract static analysis.

Result:

- no High severity issues,
- no Medium severity issues.

```
toleg@tolikk MINGW64 ~/OneDrive/Desktop/blchFinal (main)
$ ls audits
SECURITY_AUDIT_REPORT.md  slither-output.txt

toleg@tolikk MINGW64 ~/OneDrive/Desktop/blchFinal (main)
$ cat audits/slither-output.txt
```

Coverage Analysis

Coverage reports were generated using Foundry.

The project achieved extensive test coverage across protocol modules.

```
MINGW64/c/Users/toleg/OneDrive/Desktop/blchFinal
[PASS] test_governorStateNonexistentReverts() (gas: 12051)
[PASS] test_governorTimeLockAddress() (gas: 10325)
[PASS] test_governorTokenAddress() (gas: 8363)
[PASS] test_governorVotingDelayReadable() (gas: 8037)
[PASS] test_governorVotingPeriodReadable() (gas: 8001)
[PASS] test_policyAssetIsCollateral() (gas: 15399)
[PASS] test_policyDefaultAdminIsDeployer() (gas: 17664)
[PASS] test_policyNextIdStartsZero() (gas: 12950)
[PASS] test_timeLockHasGovernorProposer() (gas: 12100)
[PASS] test_timeLockOpenExecutor() (gas: 9964)
[PASS] test_treasuryImmutableTimeLock() (gas: 7985)
[PASS] test_vaultAssetIsCollateral() (gas: 15288)
[PASS] test_vaultDecimals() (gas: 18791)
[PASS] test_vaultDefaultAdminIsDeployer() (gas: 17661)
[PASS] test_vaultImplCodeSize() (gas: 5007)
[PASS] test_vaultPauserIsDeployer() (gas: 17677)
[PASS] test_vaultShareSymbol() (gas: 18056)
[PASS] test_vaultUpgraderIsDeployer() (gas: 17743)
Suite result: ok. 30 passed; 0 failed; 0 skipped; finished in 258.49ms (74.92ms CPU time)

Ran 26 tests for test/SuiteA.t.sol:SuiteATest
[PASS] testFuzz_premiumMathMatches(uint128,uint16,uint16) (runs: 256, μ: 7105, ~: 7105)
[PASS] testFuzz_premiumMonotonicInDuration(uint128,uint32,uint32) (runs: 256, μ: 6648, ~: 6648)
[PASS] testFuzz_premiumMonotonicInRate(uint128,uint16,uint16) (runs: 256, μ: 6137, ~: 6137)
[PASS] test_badOracleAnswerReverts() (gas: 371532)
[PASS] test_claimHeartbeat() (gas: 12861)
[PASS] test_claimIncreasesTotalPayouts() (gas: 429376)
[PASS] test_claimProcessorPauseBlocks() (gas: 46675)
[PASS] test_collateralSymbol() (gas: 12957)
[PASS] test_convertToAssetsMatches() (gas: 118783)
[PASS] test_expiredPolicyCannotClaim() (gas: 372125)
[PASS] test_gas_mulDivBenchmarkRecordsUsage() (gas: 1664)
[PASS] test_governorQuorumNumerator() (gas: 10502)
[PASS] test_maxDepositNonZero() (gas: 13482)
[PASS] test_maxMintNonZero() (gas: 13482)
[PASS] test_policyAnnualBps() (gas: 12987)
[PASS] test_policyPauseBlocksPurchase() (gas: 50371)
[PASS] test_policyTreasuryFeeBps() (gas: 12981)
[PASS] test_timeLockMinDelay() (gas: 7916)
[PASS] test_treasuryReceivesERC20Premiums() (gas: 333806)
[PASS] test_triggerBelowPath() (gas: 432759)
[PASS] test_upgradeVaultToV2() (gas: 3752396)
[PASS] test_vaultName() (gas: 18036)
[PASS] test_vaultPreviewDepositPositive() (gas: 28481)
[PASS] test_vaultPreviewMintRoundTrip() (gas: 28592)
[PASS] test_vaultShareBalanceOfAlice() (gas: 115776)
[PASS] test_vaultTotalAssetsTracksDeposits() (gas: 116577)
Suite result: ok. 26 passed; 0 failed; 0 skipped; finished in 716.76ms (735.75ms CPU time)

Ran 4 tests for test/ForkArbSepolia.t.sol:ForkArbSepoliaTest
[PASS] testFork_blockNumberAdvances() (gas: 6047)
[PASS] testFork_chainIdMatchesArbSepolia() (gas: 5338)
[PASS] testFork_deployMockErc20() (gas: 833178)
[PASS] testFork_readEthUsdIfConfigured() (gas: 6252)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 919.29ms (3.04s CPU time)
```

8. Deployment

Network

Ethereum Sepolia

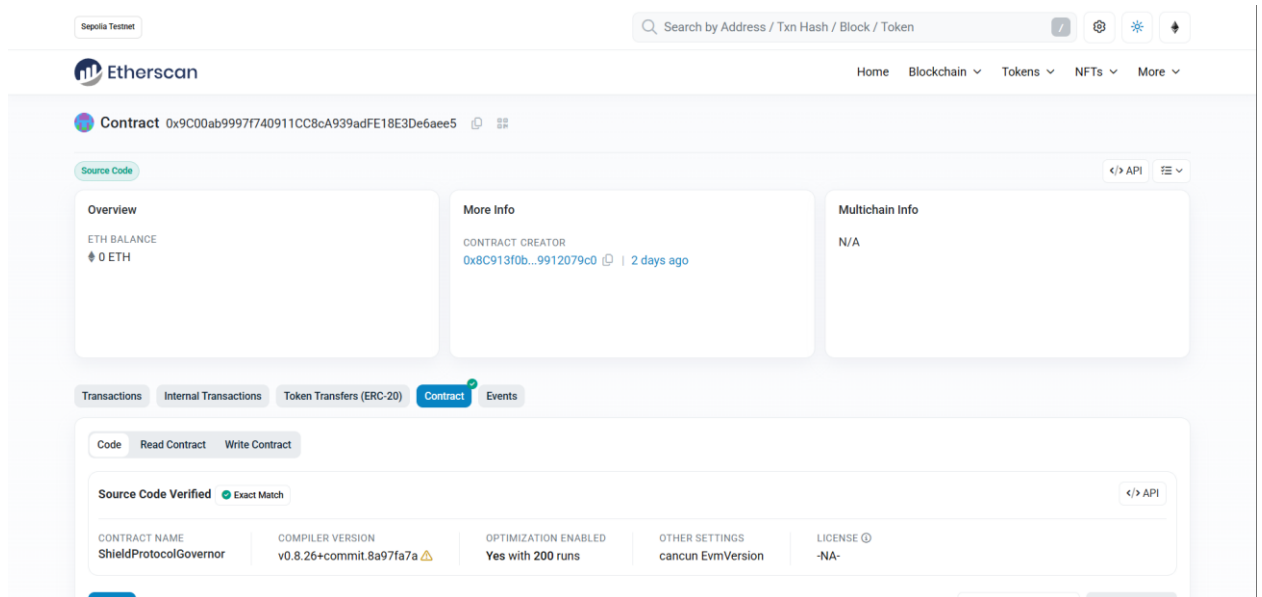
Deployed Contracts

Contract	Address
InsuranceVault	0x310b42e5092def354c30844503fba2925dccac59
PolicyManager	0x770dbcec771216667a4558deb649a51c44cc589d
ClaimProcessor	0x644635684fe8ace38a58e64f247774a40604dd56

Contract	Address
ShieldProtocolGovernor	0x9c00ab9997f740911cc8ca939adfe18e3de6aee5
ShieldGovToken	0x9944bd95587aa282e250e9ea1c8d30c6bd85b71e

Contract Verification

Contracts were verified on Sepolia Etherscan.



9. Challenges and Solutions

Challenges

- wagmi v2 transaction typing issues,
- Graph Studio deployment configuration,
- transaction gas estimation issues,
- frontend transaction error handling,
- subgraph indexing synchronization.

Solutions

- added chain and account parameters for wagmi writes,
- improved frontend readable errors,
- updated subgraph manifest configuration,
- regenerated TypeScript contract typings,
- implemented improved analytics handling.

10. Future Improvements

Potential future improvements:

- L2 deployment,
- multi-oracle support,
- dynamic premium pricing,
- NFT-based insurance positions,
- automated claim settlement,
- advanced governance proposals,
- mobile frontend support.

11. Conclusion

ShieldFi demonstrates a complete decentralized insurance protocol architecture using modern Web3 development practices.

The project successfully integrates:

- smart contracts,
- DAO governance,
- ERC4626 vaults,
- GraphQL analytics,
- frontend wallet interaction,
- security testing,
- static analysis,
- automated deployment workflows.

The final system provides a realistic example of a production-style DeFi insurance protocol.